**SCADA** *(Supervisory Control and Data Acquisition)*, **IoT** *(Internet of Things)* **Część A – bez sterownika**

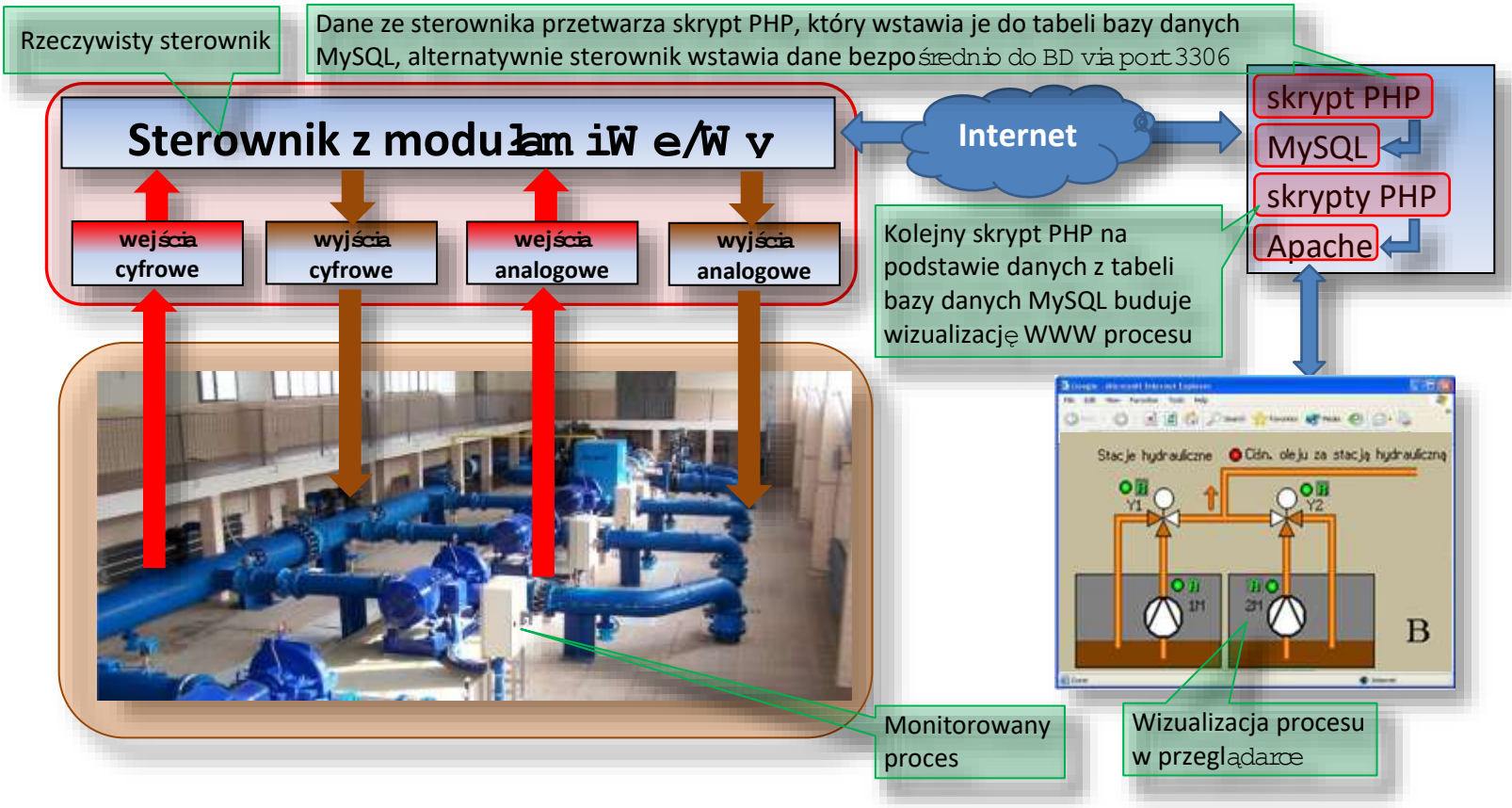
Obecnie coraz częściej zachodzi potrzeba tworzenia multimedialnych aplikacji do rejestracji, wizualizacji i zarządzania różnych procesów np. dotyczących produkcji przemysłowej, automatyki budynkowej, procesów logistycznych, przemieszczania się ludzi, pojazdów, towarów, np. w celu świadczenia określonych usług, współdzielenia określonej informacji itd. Wiele nowych technologii rozwija się dzięki twórcom takich aplikacji, dlatego warto zastanowić się nad tym, jakiej usługi dotyczącej monitorowania, rejestracji i wizualizacji brakuje na rynku i można wejść w tę niszę: zrealizować projekt w ramach niniejszego przedmiotu, następnie pracę inżynierską, a potem aplikację komercyjną.

W zależności od zastosowania oraz sposobu pracy takich systemów, składają się one z jednego lub wielu urządzeń przesyłających dane do internetowej aplikacji bazodanowej np.

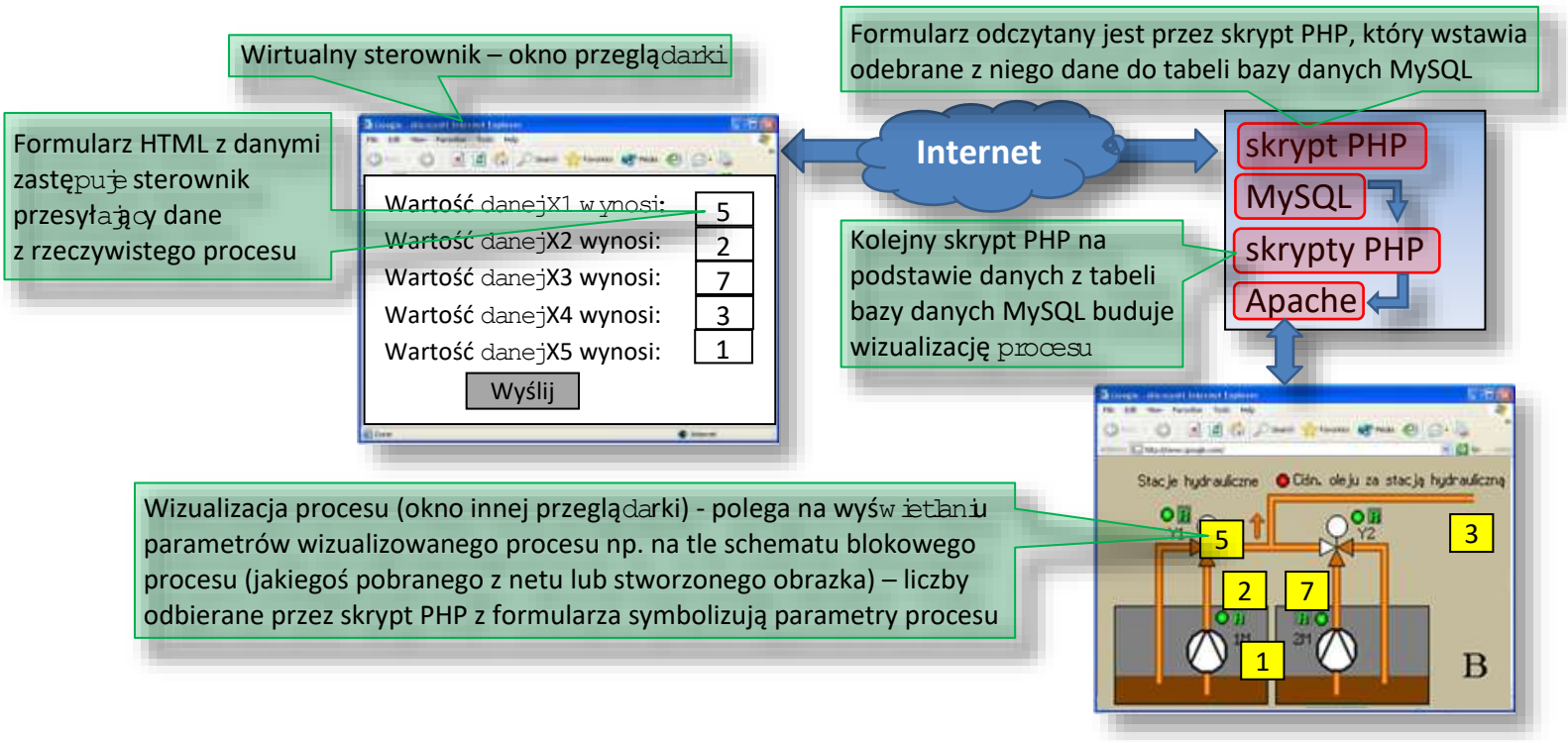
- komputery z zainstalowanymi aplikacjami np. do monitorowania ich wykorzystania, zastosowanie - do nadzorowania pracy zdalnej,
- smartfony z zainstalowanymi aplikacjami do obsługi GPS, np. do monitorowania lokalizacji terenowych pracowników socjalnych, agentów ubezpieczeniowych, pracowników zajmujących się promocją, sportowców zawodowych podczas treningów terenowych itd.,
- smartfony z zainstalowanymi aplikacjami udostępniającymi dane magazynowe firmy i obsługujące aparat/kamerę – np. do implementacji czytnika kodów kreskowych/QR, aplikacje wykorzystywane zarówno przez pracowników firm, jak i ich klientów, sterowniki PLC (**P**rogrammable **L**ogic **C**ontroller), stosowane do zarządzania procesu produkcji,
- sterowniki automatyki budynkowej, stosowane do zarządzania komfortu i bezpieczeństwem w nowoczesnych budynkach (temperatura, wilgotność, wentylacja, oświetlenie, windy, p.poż., SSWiN, itd.),
- różne typy mikrokontrolerów z interfejsem sieciowym np. Raspberry Pi, Arduino z Ethernet Shield, STM32 itd., współpracujące z sensorami różnych wielkości fizycznych i elektrycznych oraz elementami wykonawczymi, wyświetlaczami itd.

Wymienione urządzenia monitorują stan określonego procesu i na jego podstawie generują określone dane np. dotyczące położenia GPS i innych parametrów np. szybkości poruszania się. Przykładami takich aplikacji są Google Maps / Apple Maps, które na podstawie informacji o lokalizacji GPS urządzenia, podpowiadają kierunek poruszania się, aby osiągnąć cel podróży zadany przez użytkownika. Dodatkowo aplikacje takie mogą informować o korkach, na podstawie danych o prędkości poruszania się innych klientów tej aplikacji.

W przypadku systemów automatyki przemysłowej lub budynkowej sterowniki PLC lub specjalistyczne sterowniki automatyki budynkowej są źródłem informacji do systemu nadrzędnego i jednocześnie miejscem, do którego kierowane są dane z tego systemu. Sterowniki te zwykle komunikują się z systemem nadrzędnym typu SCADA (https://pl.wikipedia.org/wiki/SCADA), poprzez sieć LAN lub/i Internet. Najtańsza implementacja takiego sterownika składa się z mikrokontrolera Arduino + Ethernet Shield lub Raspberry Pi. Poniżej przedstawiono uproszczony schemat blokowy takiego systemu, wykorzystującego technologię PHP/MySQL. Sterownik (poprzez sieć Internet) przesyła dane z monitorowanego procesu do serwera na hostingu. Na tym serwerze odpowiedni skrypt PHP umieszcza odebrane dane w bazie MySQL. Serwer przetwarza dane i wypracowuje odpowiednie sygnały sterujące, które następnie przesyła do sterownika.
IoT - https://pl.wikipedia.org/wiki/Internet_rzeczy



Zbudowanie powyższego systemu, z uwzględnieniem sterownika monitorującego jakiś proces, przekracza czas pojedynczych zajęć laboratoryjnych. Dlatego zadanie w pierwszej **Części A** zostanie uproszczone: sterownik zostanie zastąpiony przez przeglądarkę internetową z formularzem umożliwiającym wstawianie danych. Schemat blokowy uproszczonego systemu przedstawiono poniżej.




**Wytyczne do realizacji zadania**

**Uwaga:** zadanie składa się z wielu elementów, dlatego aby prowadzący mógł je szybko sprawdzić, należy w katalogu przechowującym skrypty z tego zadania stworzyć aplikację w środowisku Bootstrap lub innym środowisku umożliwiającym osadzenie poleceń w menu, aby obsługa aplikacji była prosta i intuicyjna.

**Generowanie wykresów w PHP**

Istotnym elementem każdego systemu wizualizacji jest możliwość przedstawiania danych w postaci parametryzowanych wykresów. Taka forma pozwala na błyskawiczną orientację w zmianach danych zachodzących w czasie. O wiele łatwiej zauważyć wzrosty lub spadki określonych parametrów procesu na wykresie, niż poprzez analizę danych przedstawianych w tabeli.
Nie bez racji ludzie powtarzają: *One picture is worth a thousand words*.

Poniżej przedstawiono kilka przykładowych rozwiązań:

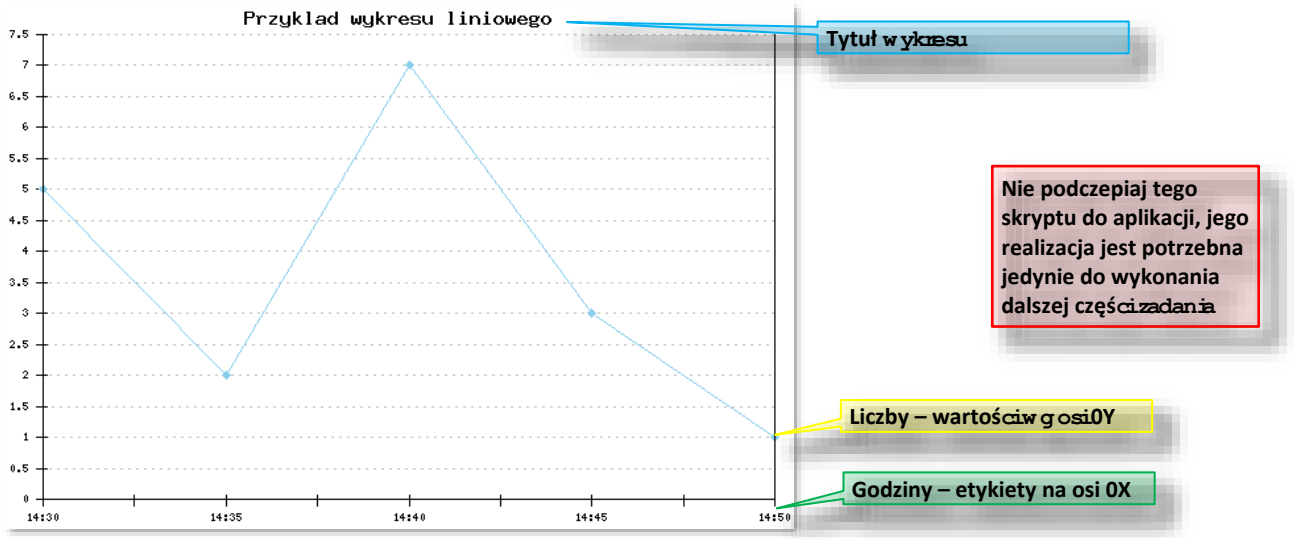
a. W Internecie można znaleźć wiele prostych, a także bardziej złożonych skryptów, umożliwiających tworzenie wykresów z danych pobranych z tablicy PHP lub tabeli bazy danych m.in.:
   i. phplot http://sourceforge.net/projects/phplot
   ii. libchart https://naku.dohcrew.com/libchart/pages/introduction/
   iii. jpGraph http://jpgraph.net
   iv. phpchart http://phpchart.com
   v. XML/SWF Charts http://www.maani.us/xml_charts

b. Aby wykres mógł być dynamicznie generowany na Twoim serwerze - ściągnij z Internetu i umieść w podkatalogu serwera hostingowego przeznaczonym na to zadanie, pliki wybranego skryptu do tworzenia wykresów, jeśli zdecydowałeś się np. na użycie phplot:
   i. ze strony http://sourceforge.net/projects/phplot ściągnij spakowaną paczkę plików i wypakuj z niej skrypt **phplot.php** oraz umieść go na swoim hostingu, w tym samym katalogu, w którym umieścisz plik generujący wykres,
   ii. na stronie http://phplot.sourceforge.net/phplotdocs znajdziesz informacje niezbędne do prawidłowej instalacji i wykorzystania skryptu, są tam również przykłady umożliwiające przegląd dostępnych typów wykresów.
   iii. Stwórz plik **wykres.php** - zawierający poniższy kod generujący przykładowy wykres:

**wykres.php**

```
<?php
require_once 'phplot.php';
$plot = new PHPlot();
$data = array(array('14:30',0,5), array('14:35',1,2), array('14:40',2,7), array('14:45',3,3), array('14:50',4,1));
$plot -> SetDataValues($data);
$plot -> SetDataType('data-data');
$plot -> SetTitle('Przyklad wykresu liniowego');            // opcjonalny tytuł wykresu
$plot -> DrawGraph();
?>
```

iv. Przetestuj działanie powyższego skryptu - powinien wyświetlić się wykres pokazany poniżej.
Jeśli się nie wyświetla – prawdopodobnie brak pliku phplot.php w katalogu uruchamianego skryptu **wykres.php**.




**v.** Docelowo wizualizacja musi zawierać m.in. wykresy danych pobieranych z tabeli bazy danych MySQL przechowywanych na hostingu, a nie dane z tablic PHP. Zatem, jeśli chcesz wykorzystać powyżej przedstawiony kod – będziesz musiał zastąpić podane w nim liczby zaznaczone markerem - za pomocą zmiennych PHP, które wcześniej będziesz musiał załadować danymi z tabeli bazy danych.

**vi.** W Internecie znajduje się również mnóstwo przykładów kodu do generowania wykresów z bezpośrednim pobieraniem danych z tabeli bazy danych.
https://www.google.com/search?rlz=1C1CHBD_enPL851PL866&q=PHP+charts+from+MySQL+free&sa=X&ved=2ahUKEwiEkfTvub72AhXw-SoKHW8pCY4Q1QJ6BAglEAE

**Właściwa część aplikacji**

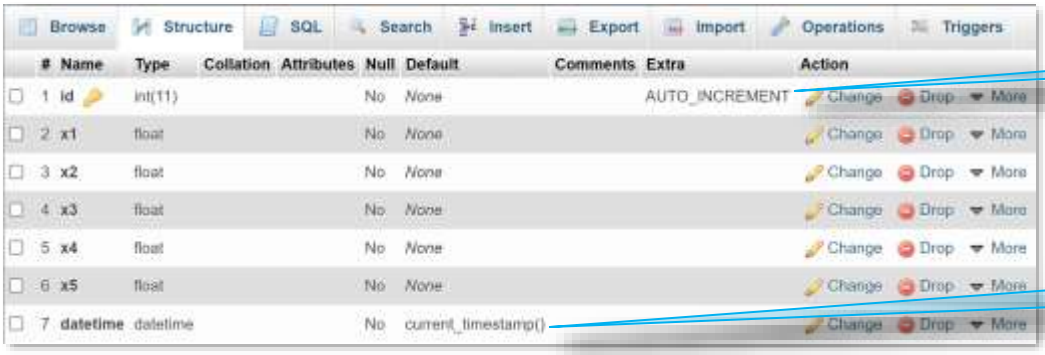
1. Ogólne informacje o aplikacji: tak, jak przedstawia drugi rysunek na drugiej stronie instrukcji – aplikacja składa się z dwóch części:

- **Pierwsza część** (okno przeglądarki np. Google Chrome)
  - za pomocą przedstawionego na rysunku formularza przesyła 5 liczb do skryptu PHP na serwerze,
  - skrypt PHP na serwerze wstawia 5 liczb odebranych z formularza - do tabeli bazy danych MySQL.

- **Druga część** (okno przeglądarki np. Firefox)
  - wizualizacja pracy kontrolowanego systemu, na podstawie danych z tabeli bazy danych MySQL, tzn. wyświetla dane pobierane z tabeli bazy danych na tle rzutu parteru budynku stworzonego w ArchiCADzie (format SVG),
  - aktualizuje wyświetlane dane z wykorzystaniem technologii AJAX, SSE lub Websocket, aby umożliwić obserwowanie zmian w tabeli bazy danych, które następują po wprowadzeniu nowych danych przez formularz w oknie drugiej przeglądarki,
  - początkowo można to zrobić przez cykliczne przeładowania strony – przez dodanie kodu do <head>:
    <head> <meta http-equiv="refresh" content="10" /> </head>,
    jednak finalnie należy wykorzystać technologię: AJAX, SSE lub Websocket.

2. Przejdź do phpMyAdmin'a i stwórz tabelę o nazwie ***pomiary*** zawierającą 7 kolumn: pomiary (**id**, x1, x2, x3, x4, x5, datetime)
   a. id - klucz główny z autonumerowaniem,
   b. x1, x2, x3, x4, x5 - pięć danych typu float,
   c. datetime – data/godzina, wartość domyślna CURRENT_TIMESTAMP (czas utworzenia rekordu).

Tabela **pomiary**



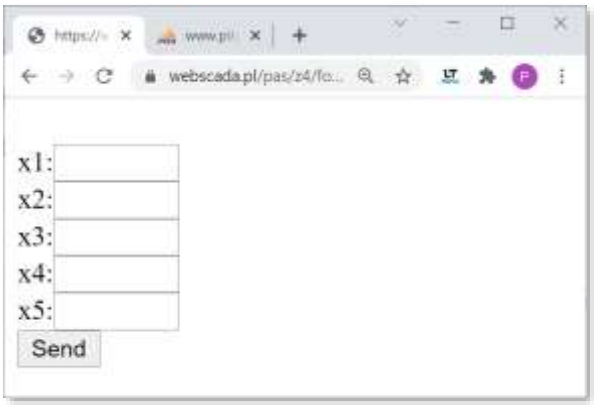

3. Stwórz plik o nazwie **formularz.php**, pozwalający na przesyłanie danych (x1, x2, x3, x4, x5) do serwera MySQL.
   a. W pierwszej fazie stwórz formularz przedstawiony poniżej i przetestuj jego działanie.
      Zweryfikuj, czy wymusza on przesyłanie wyłącznie liczb ze wskazanego zakresu i uniemożliwia wysyłanie np. liter.
   b. Zwróć uwagę, że formularz w tej postaci może wysyłać puste wartości, co generuje błędy po stronie skryptu add.php.
   c. Docelowo zmodyfikuj formularz, aby nie było możliwe wysyłane pustych pól.

**formularz.php**

```
<form method="POST" action="add.php"><br>
   x1:<input type="number" name="x1" min="-50" max="100"><br>
   x2:<input type="number" name="x2" min="-50" max="100"><br>
   x3:<input type="number" name="x3" min="-50" max="100"><br>
   x4:<input type="number" name="x4" min="-50" max="100"><br>
   x5:<input type="number" name="x5" min="-50" max="100"><br>
   <input type="submit" value="Send"/>
</form>
```

**type ="number", min="…" i max="…" - umożliwiają ograniczenie zakresu wprowadzanych liczb**

Formularz powinien wyglądać mniej więcej tak:




faktyczny wygląd formularza zależy od typu przeglądarki

4. Stwórz skrypt **add.php**, odbierający dane z formularza i dodający je do tabeli **pomiary**
   a. W BD będzie pojawiał się kolejny rekord każdorazowo po wysłaniu danych z formularza.
   b. Przetestuj działanie plików **formularz.php** i **add.php**, wysyłając do serwera zawartość kilku formularzy z różnymi danymi i sprawdź, czy dane te pojawiają się, jako kolejne rekordy w tabeli pomiary.
   c. Zwróć uwagę, że poniższy plik nie jest w żaden sposób zabezpieczony przed błędami np. wysłaniem pustego formularza, błędnymi parametrami BD itd. Zabezpiecz skrypt, aby wyeliminować większość potencjalnych błędów eksploatacyjnych.

**add.php**

```
<?php
        $x1 = $_POST['x1'];
        $x2 = $_POST['x2'];
        $x3 = $_POST['x3'];
        $x4 = $_POST['x4'];
        $x5 = $_POST['x5'];
        $dbhost="localhost"; $dbuser="Your_dbuser"; $dbpassword="Your_dbpassword"; $dbname="Your_dbname";
        $polaczenie = mysqli_connect($dbhost, $dbuser, $dbpassword, $dbname);
        $result = mysqli_query($polaczenie, "INSERT INTO pomiary (x1,x2,x3,x4,x5) VALUES ($x1,$x2,$x3,$x4,$x5)");
        mysqli_close($polaczenie);
        header('Location: index.php');
?>
```

**Dane odebrane z formularza**

**Wstaw dane z Twojego hostingu**

**Przejście do pliku menu index.php**

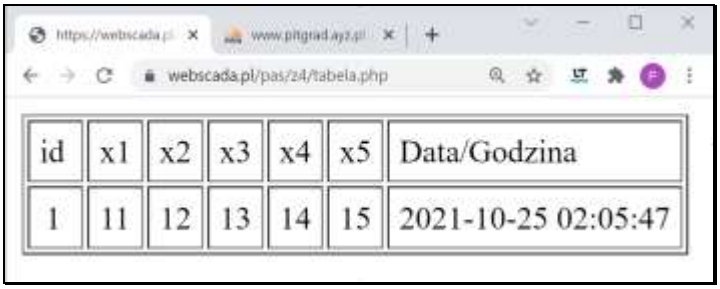
5. Stwórz plik **tabela.php**, wyświetlający zawartość tabeli pomiary.

**Wstaw dane z Twojego hostingu**

**tabela.php**

```
<?php
    $dbhost="localhost"; $dbuser="Your_dbuser"; $dbpassword="Your_dbpassword"; $dbname="Your_dbname";
    $polaczenie = mysqli_connect($dbhost, $dbuser, $dbpassword, $dbname);
    $rezultat = mysqli_query($polaczenie, "SELECT * FROM pomiary");
    print "<table cellpadding=5 border=1>";
    print "<tr><td>id</td><td>x1</td><td>x2</td><td>x3</td><td>x4</td><td>x5</td><td>Data/Godzina</td></tr>\n";
    while ($wiersz = mysqli_fetch_array ($rezultat))
    {
        $id = $wiersz[0];
        $x1 = $wiersz[1];
        $x2 = $wiersz[2];
        $x3 = $wiersz[3];
        $x4 = $wiersz[4];
        $x5 = $wiersz[5];
        $datetime = $wiersz[6];
        print "<tr><td>$id</td><td>$x1</td><td>$x2</td><td>$x3</td><td>$x4</td><td>$x5</td><td>$datetime</td></tr>\n";
    }
    print "</table>";
    mysqli_close($polaczenie);
?>
```

Tabela po wprowadzeniu zawartości jednego formularza powinna wyglądać mniej więcej tak:




faktyczny wygląd tabeli zależy od typu przeglądarki

6. Zmodyfikuj aplikację w taki sposób, aby w postaci wykresu wyświetlać liczby z ostatniego rekordu w tabeli **pomiary**.
   Jeśli zamierzasz wykorzystać w dalszej części zadania skrypt **phplot.php**, to możesz to zrobić tak, jak to sugeruje poniższa instrukcja.
   a. Skopiuj po kolei obydwa skrypty **tabela.php** oraz **wykres.php** do nowego pliku **wykres2.php**.
   b. W części pliku **wykres2.php**, który oryginalnie pochodzi z pliku **wykres.php** - zamień liczby reprezentujące wartości na osi 0Y za pomocą zmiennych $x1, $x2, $x3, $x4, $x5, pobieranych z tabeli **pomiary**, czyli z części pliku **wykres2.php**, który oryginalnie pochodzi z pliku **tabela.php**.

**wykres2.php** (fragment)

```
…
$data = array( array('Tytuł',0,$x1),array('',1,$x2),array('',2,$x3),array('',3,$x4), array('',4,$x5),);
…
```

**Nie podczepiaj tego skryptu do aplikacji, jego realizacja jest potrzebna jedynie do wykonania dalszej części zadania**

   c. Podsumowując: zmodyfikuj skrypt **wykres2.php** w taki sposób, aby wykres był rysowany na podstawie ostatniego rekordu z tabeli **pomiary**, tip: SELECT * FROM pomiary ORDER BY id DESC LIMIT 1;
   d. Przetestuj, czy wykres każdorazowo generuje się na podstawie ostatnio przesłanych danych z formularza.

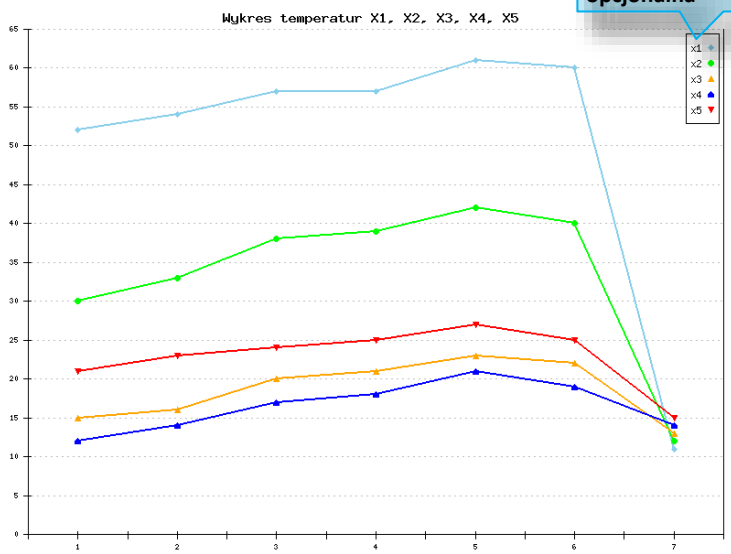
7. Skopiuj skrypt **wykres2.php** do nowego pliku **wykres3.php**.
   a. Zmodyfikuj skrypt **wykres3.php** tak, aby działał on, jak jest to przedstawione na rysunku poniżej.
   b. Chodzi o to, aby odtwarzać 5 serii danych wprowadzonych przez formularz, gdzie każda seria odpowiada kolejnej zmiennej
      i. pierwsza seria odpowiada kolejnym wartościom przypisanym do zmiennej $x1,
      ii. druga seria odpowiada kolejnym wartościom przypisanym do zmiennej $x2,
      iii. trzecia seria odpowiada kolejnym wartościom przypisanym do zmiennej $x3,
      iv. czwarta seria odpowiada kolejnym wartościom przypisanym do zmiennej $x4,
      v. piąta seria odpowiada kolejnym wartościom przypisanym do zmiennej $x5,

**To jest właściwa część zadania do realizacji, jeśli chodzi o tworzenie wykresu**

   c. Np. jeśli pierwszy wysłany formularz zawierał liczby 2,4,6,8,10, drugi formularz 3, 5, 7, 9, 11, a trzeci 1, 3, 5, 7, 9, to będzie można zaobserwować, że wartości wszystkich kolejnych zmiennych $x1, $x2, $x3, $x4, $x5 zwiększyły się najpierw o 1, a w kolejnej serii zmniejszyły się o 2. Oczywiście zwykle się nie zdarza, aby dane w poszczególnych seriach różniły się o tę samą wartość, tutaj jednak został przytoczony taki szczególny przypadek, aby zobrazować różnice między seriami danych.
   d. **Dodatkowe objaśnienie:** tym razem wykres nie ma przedstawiać jednej krzywej zbudowanej z odcinków przechodzących przez liczby pochodzące z ostatniego formularza tzn. pięciu ostatnio wprowadzonych liczb – tak jak to miało miejsce w poprzednio. Tym razem chodzi o to, aby wyświetlić wykres składający się z pięciu krzywych, zbudowanych z serii danych pobieranych odpowiednio ze zmiennych $x1, $x2, $x3, $x4, $x5. Należy zatem wyświetlić **wszystkie dane** zgromadzone w tabeli **pomiary**. Gdy zostanie przesłana zawartość kolejnego formularza, wówczas w tabeli pojawi się kolejny rekord, a każda z krzywych na wykresie wzbogaci się o kolejny punkt po prawej stronie wykresu.
   e. Poniższy rysunek przedstawia tabelę zawierająca 7 rekordów, powstałych przez wysłanie zawartości siedmiu formularzy. Obok znajduje się wykres zbudowany na podstawie zawartości tych formularzy. Kolory ułatwiają orientację.

**Legenda jest opcjonalna**

| id | x1 | x2 | x3 | x4 | x5 | Data/Godzina |
|---|---|---|---|---|---|---|
| 1 | 52 | 30 | 15 | 12 | 21 | 2021-11-28 16:41:49 |
| 2 | 54 | 33 | 16 | 14 | 23 | 2021-11-28 16:43:46 |
| 3 | 57 | 38 | 20 | 17 | 24 | 2021-11-28 16:45:30 |
| 4 | 57 | 39 | 21 | 18 | 25 | 2021-11-28 16:46:31 |
| 5 | 61 | 42 | 23 | 21 | 27 | 2021-11-28 16:47:30 |
| 6 | 60 | 40 | 22 | 19 | 25 | 2021-11-28 16:47:55 |
| 7 | 11 | 12 | 13 | 14 | 15 | 2021-12-13 15:38:48 |




   f. Jeśli nie wiesz jak zmodyfikować skrypt rysujący wykres – zajrzyj ponownie na stronę, gdzie przedstawiono informacje, jak prawidło wykorzystywać skrypt do wyświetlania kilku serii danych http://phplot.sourceforge.net/phplotdocs.
   g. Jeśli zdecydowałeś się na innych skrypt – poszukaj podpowiedzi w dokumentacji tego skryptu.

8. Stwórz wizualizację ogrzewania budynku stworzonego w ArchiCADzie, wykorzystując dane zgromadzone w tabeli **pomiary**:
   a. Stwórz plik **scada.php** i umieść w tle plik SVG rzutu parteru budynku stworzonego w ArchiCADzie oraz obrazek kotła grzewczego.
   b. Aktualne temperatury (ostatni rekord z tabeli **pomiary**) muszą się wyświetlać na wizualizacji tego budynku.
   tip: SELECT * FROM pomiary ORDER BY id DESC LIMIT 1;
   c. Umieść po prawej stronie wykres przebiegu 5 temperatur, uwzględniający wartości aktualne (będą one wyświetlane w zależności od konfiguracji skryptu na początku lub na końcu wykresu) oraz dane historyczne.
   d. Przetestuj, czy dodawanie kolejnych danych z formularza zwiększa ilość danych wyświetlanych na wykresie.

Zamiast symboli X1, X2, X3, X4, X5 mają być wyświetlane liczby pochodzące z ostatniego rekordu z tabeli **pomiary**, ponieważ wizualizacja ta ma przedstawiać aktualną sytuację

Efekt działanie skryptu **scada.php**

**Rzut parteru budynku stworzonego w ArchiCADzie** **Kocioł ogrzewania** **Wykres temperatur X1, X2, X3, X4, X5**

9. Rozbuduj skrypt **scada.php** o elementy indywidualne, charakterystyczne dla Twojego projektu: w zależności od grupy należało zrealizować odpowiedni projekt budynku w ArchiCADzie m.in. bydynek biurowy firmy IT, Datacenter itd.

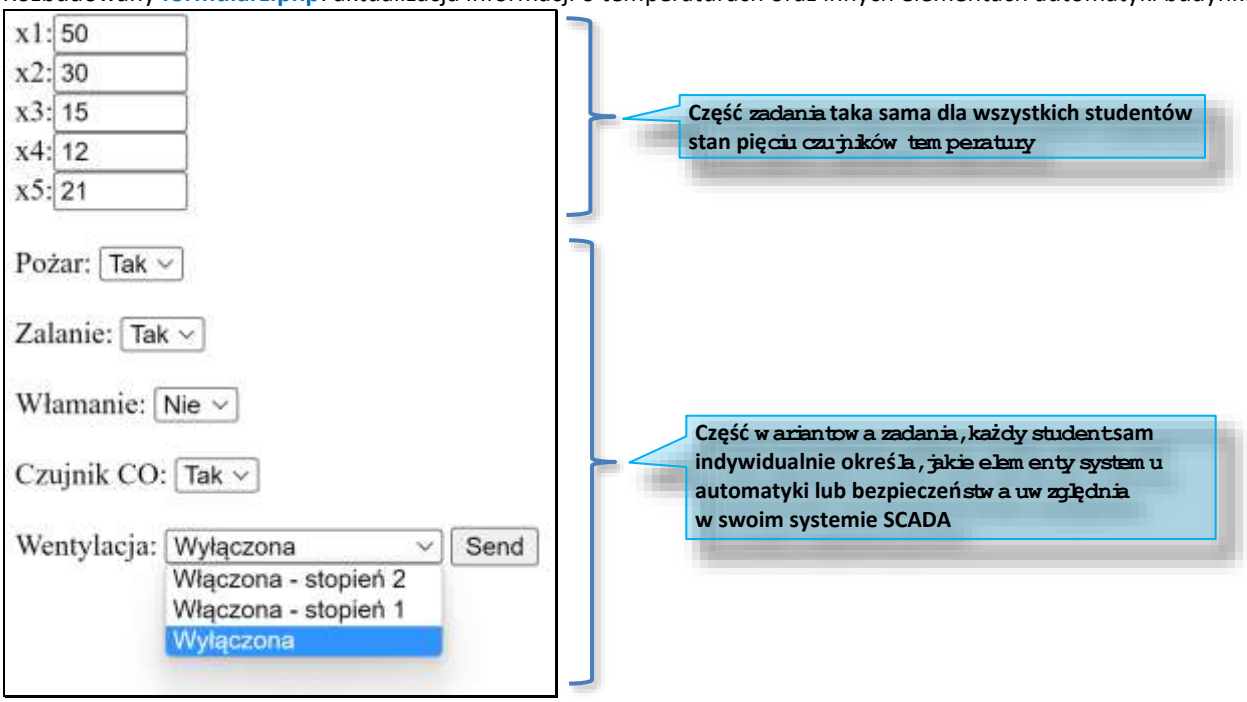
   a. Obowiązkowymi elementami wizualizacji jest 5 czujników temperatury, które mają dotyczyć (w przypadku firmy IT):
      i. X1 Temperatury zasilania grzejników (czerwona strzałka),
      ii. X2 Temperatury powrotu z grzejników (niebieska strzałka),
      iii. X3 Temperatury w pierwszym pomieszczeniu biurowym.
      iv. X4 Temperatury w drugim pomieszczeniu biurowym.
      v. X5 Temperatury w trzecim pomieszczeniu biurowym.

   b. W przypadku Datacenter należy wszystkie czujniki umieścić w kolejnych rzędach szaf serwerowych.

   c. Poniżej wymieniono wariantowe elementy wizualizacji: statyczne i animowane ikony symbolizujące pracę lub awarię systemu wentylacji, klimatyzacji, ogrzewania, systemu kontroli dostępu, sygnalizacji włamania i napadu, sygnalizacji pożarowej, czujnika zalania, czujnika ulatniania się gazu, czujnika $CO_2$, czujnika CO itd.
      i. Przykładowe ikony dotyczące tych systemów zamieszczono na końcu zadania.
      ii. W Internecie znajdują się gotowe pliki SVG przedstawiające w/w symbole, nie ma żadnego uzasadnienia, aby je tworzyć, chyba, że ktoś odczuwa potrzebę tworzenia ikon SVG ☺.
      iii. W momencie, gdy następuje „aktywacja” wybranego systemu np. wybucha pożar, następuje zalanie, ulatnia się gaz - wybrane ikony SVG należy animować, można skorzystać z doświadczeń wyniesionych z zadania dotyczącego SVG lub https://www.w3schools.com/graphics/svg_animation.asp
      https://docs.fontawesome.com/web/style/animate
      https://codepen.io/mowcode/pen/PvEWPN

d. Nie trzeba wstawiać wszystkich wymienionych czujników, należy wybrać zestaw 5 dodatkowych elementów wizualizacji i umieścić je na planie wizualizacji oraz rozbudować o nie również **formularz.php**, umożliwiający symulowanie działania sterownika, który przekazuje na serwer stan różnych czujników, dzięki czemu można obserwować przez Internet nie tylko rozkład temperatur w budynku, ale także dowiedzieć się czy działa wentylacja, klimatyzacja, ogrzewanie, czy jest zazbrojony alarm, czy się pali lub czy biuro jest zalane lub czy jest zbyt wysoki poziom gazu ziemnego, $CO_2$ lub CO.
**UWAGA:** Aby wszystko działało poprawnie należy również odpowiednio rozbudować plik **add.php** oraz tabelę **pomiary**.

Rozbudowany **formularz.php**: aktualizacja informacji o temperaturach oraz innych elementach automatyki budynku




e. W zależności od danych w bazie danych powinny się wyświetlać ikony statyczne lub animowane np. jeśli wentylacja jest włączona na 100% - wyświetla się w odpowiednim miejscu wizualizacji szybko ruszający się wiatraczek, jeśli na 50% - wówczas odpowiednio wolniej, jeśli wentylacja jest wyłączona – wówczas wyświetlana jest statyczna ikona.
Dla wentylacji można zatem przyjąć 3 stany pracy: 0, 1 i 2, które będą wysyłane do bazy danych przez formularz.
   i. stan 0 – wentylator nie kręci się,
   ii. stan 1 – wentylator kręci się wolno,
   iii. stan 2 – wentylator kręci się szybko.

f. Dla wielu innych elementów wystarczą dwa stany: 0 i 1, np. czujnik pożaru:
   i. stan 1 wykryty pożar – np. wyświetla się animowany płomień np. pulsujący,
   ii. stan 0 - brak pożaru - np. wyświetla się statyczna ikona gaśnicy, to znaczy, że system czuwa, ale wszystko jest Ok.

10. Dodaj w wizualizacji półprzezroczystą warstwę nad pomieszczeniami budynku (w przypadku Datacenter nad określoną częścią szaf rackowych), o kolorze uzależnionym od temperatury w danym pomieszczeniu/szafie rackowej:
   a. **niebieski** $T < 10^0C$,
   b. **zielony** $10^0C <= T < 20^0C$,
   c. brak warstwy $20^0C <= T < 25^0C$,
   d. **pomarańczowy** $25^0C <= T < 30^0C$,
   e. **czerwony** $T >= 30^0C$.
   W przypadku Datacenter:
   a. **niebieski** $T < 30^0C$,
   b. **zielony** $30^0C <= T < 40^0C$,
   c. brak warstwy $40^0C <= T < 50^0C$,
   d. **pomarańczowy** $50^0C <= T < 60^0C$,
   e. **czerwony** $T >= 60^0C$.

11. Poniżej przedstawiono m.in. linki do wskaźników i innych elementów graficznych od firmy Google, która bardzo wspiera technologię IoT. Wykorzystaj przynajmniej dwa przykłady elementy graficzne firmy Google w swojej wizualizacji.
    https://developers.google.com/chart/interactive/docs/examples
    https://developers.google.com/chart/interactive/docs/gallery/gauge

12. Dodaj do aplikacji rejestrację informacji geolokalizacyjnych o gościach portalu i szczegółowych informacji o przeglądarce, rozdzielczości, kolorach, ciasteczkach itd. Umieść w menu link umożliwiający podgląd tych informacji.

**Przykładowe ikony związane z wizualizacją**

Wentylacja

Chłodzenie (Klimatyzacja)

Ogrzewanie

Alarm systemu kontroli dostępu

Alarm systemu sygnalizacji włamania i napadu

Alarm systemu sygnalizacji pożarowej

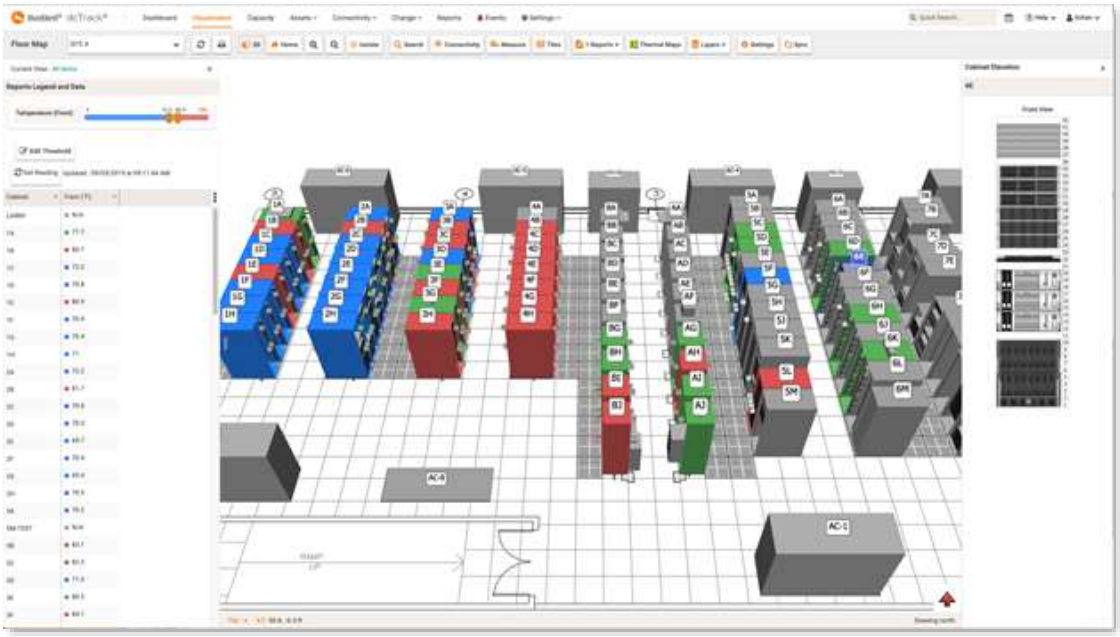
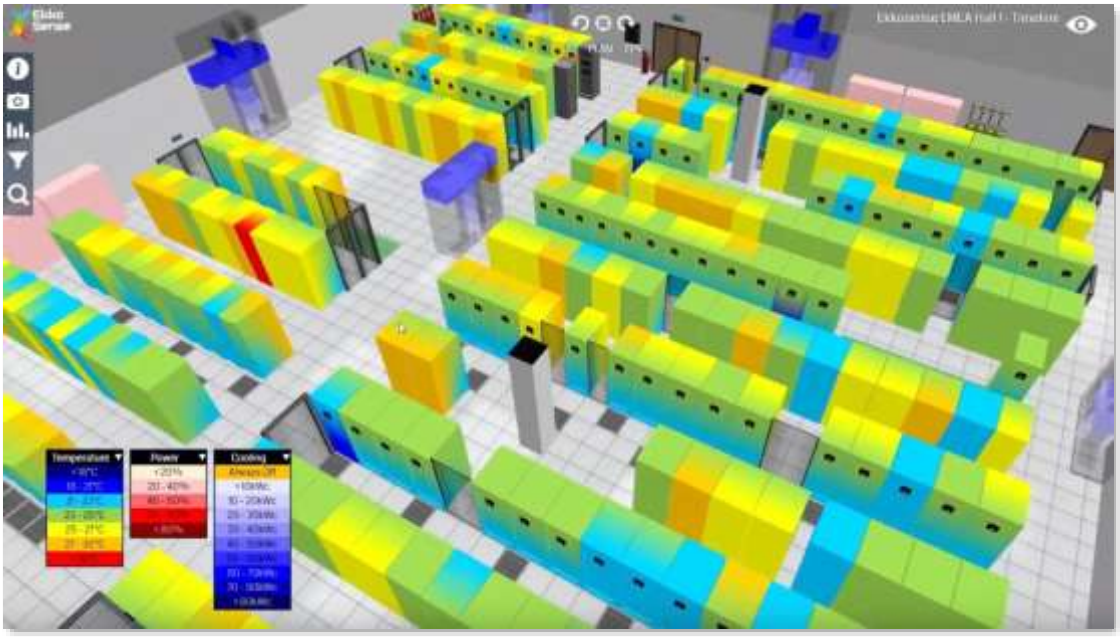
Czuwanie systemu sygnalizacji pożarowej (brak alarmu)

Alarm wycieku wody

Przekroczony dopuszczalny poziom gazu ziemnego (wyciek)

Przekroczony dopuszczalny poziom $CO_2$

Przekroczony dopuszczalny poziom CO

**Uwagi dodatkowe:**

- W wielu praktycznych zastosowaniach zamiast liczbowych informacji o temperaturach, wyświetla się określone elementy graficzne wizualizacji w odpowiednio dobranych kolorach, ma to sens szczególnie jeśli dotyczy to np. szaf rackowych o monitorowanej temperaturze
    - kolor **niebieski** i **zielony** – niska temperatura,
    - kolor **biały** i **żółty** – neutralna temperatura,
    - kolor **pomarańczowy** i **czerwony** – zbyt wysoka temperatura,
    - kolor **szary** – jeśli jakiś element nie jest monitorowany, nie działa lub nie jest podłączony.